MLOps Heart Disease Prediction — Project Report

Course: MLOps (S2-25_AMLCSZG523) — Assignment I **Dataset:** UCI Heart Disease (Cleveland), 303 rows × 14 attributes **Repository:** https://github.com/uday121212/mlops-heart-disease

---

1. Setup & Install Instructions

```
git clone https://github.com/uday121212/mlops-heart-disease.git
cd mlops-heart-disease
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt

python -m src.download_data    # fetch + clean dataset
python -m src.train            # train, log to MLflow, persist best mod
pytest                         # run unit + API tests
uvicorn src.api:app --reload   # serve locally on :8000
```

Docker:

```
docker build -t heart-disease-api:latest .
docker run -p 8000:8000 heart-disease-api:latest
```

Compose stack (API + Prometheus + Grafana):

```
docker compose up --build
```

Kubernetes (Helm):

```
helm install heart-api ./helm/heart-api
```

---

2. Data Acquisition & EDA

The raw `processed.cleveland.data` file is fetched from the UCI ML Repository by `src/download_data.py`. Cleaning steps:

- Replace `?` markers with `NaN`.
- Median-impute the few missing values in `ca` and `thal`.
- Binarize `num` (0 / 1–4) into `target` (0 = no disease, 1 = disease).

Highlights from the EDA notebook (notebooks/01_eda.ipynb):

- Class balance is roughly 54% / 46% (no resampling required).
- Age distribution: mean ≈ 54, std ≈ 9.
- Strongest correlations with `target`: `cp`, `thalach` (negative), `oldpeak`, `ca`, `exang`.
- No severe multicollinearity detected.

Generated figures: histograms (`reports/hist_*.png`), correlation heatmap (`reports/corr_heatmap.png`), class-balance plot (`reports/class_balance.png`).

---

3. Feature Engineering & Model Development

Features split into:

- **Numeric:** `age`, `trestbps`, `chol`, `thalach`, `oldpeak`, `ca` → `StandardScaler`.
- **Categorical:** `sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `thal` → `OneHotEncoder(handle_unknown="ignore")`.

Both are wrapped in a single `ColumnTransformer` so that the same preprocessing ships with the model (full reproducibility — no leakage between train and test).

Models compared (5-fold stratified CV, scoring = `roc_auc`):

Model	Hyper-parameter grid
Logistic Regression	`C ∈ {0.1, 1, 10}`, `penalty ∈ {l1, l2}`
Random Forest	`n_estimators ∈ {200, 400}`, `max_depth ∈ {None, 5, 10}`, `min_samples_split ∈ {2, 5}`

The best model (highest CV ROC-AUC) is persisted to `models/heart_disease_model.joblib`. Test-set metrics from a recent run are written to `reports/metrics.json` and include accuracy, precision, recall, F1, and ROC-AUC.

---

4. Experiment Tracking — MLflow

Every grid-search run is logged via MLflow with:

- Parameters (best params per model).
- Metrics (CV ROC-AUC, test accuracy / precision / recall / F1 / ROC-AUC).
- Artifacts (the fitted `Pipeline` via `mlflow.sklearn.log_model`).

Tracking URI: `file://./mlruns`. Browse with:

```
mlflow ui --backend-store-uri ./mlruns
```

Screenshots: screenshots/mlflow-runs.png, screenshots/mlflow-experiment.png.

---

5. Model Packaging & Reproducibility

- `requirements.txt` pins runtime deps; `requirements-dev.txt` adds tooling.
- The serialized model is the **full `Pipeline`** (preprocessor + estimator), so inference cannot drift from training.
- The same `build_pipeline` factory is used by training and tests, guaranteeing identical preprocessing.

---

6. CI/CD Pipeline

`.github/workflows/ci.yml` defines two jobs:

1. **lint-test-train**: flake8 → download data → train model → pytest with coverage → upload model + coverage artifacts.
2. **docker-build**: pulls the trained model artifact, builds the Docker image with Buildx, runs the container, and smoke-tests `/health` + `/predict`.

The pipeline fails on any lint, test, or training error, with logs surfaced in the Actions UI. Screenshot: screenshots/ci.png.

---

7. Containerization

`Dockerfile` (Python 3.11-slim, multi-step):

- Installs pinned dependencies.
- Copies `src/` and `models/`.
- Runs as a non-root `appuser`.
- Exposes port 8000, ships a `HEALTHCHECK`, and starts `uvicorn`.

`docker-compose.yml` orchestrates API + Prometheus + Grafana with provisioned data source and dashboard.

---

8. Production Deployment (Kubernetes)

Manifests in [k8s/](k8s/):

- `deployment.yaml` — 2 replicas, readiness/liveness probes on `/health`, resource requests/limits, Prometheus scrape annotations.
- `service.yaml` — LoadBalancer on port 80 → container 8000.
- `ingress.yaml` — Optional NGINX ingress on `heart-api.local`.

Helm chart in helm/heart-api/ parameterizes image, replicas, service type, ingress, and HPA.

Quick deploy on Docker Desktop / Minikube:

```
docker build -t heart-disease-api:latest .
kubectl apply -f k8s/
kubectl port-forward svc/heart-api 8080:80
curl http://localhost:8080/health
```

API screenshots: screenshots/swagger-ui.png, screenshots/api-metrics.png. Capture `kubectl get pods/svc` and a `curl /predict` response into `screenshots/k8s-pods.png`, `screenshots/k8s-service.png`, `screenshots/predict.png` after deploying.

---

9. Monitoring & Logging

- The API uses Python `logging` for structured per-request logs (level, timestamp,

prediction).

- `prometheus-fastapi-instrumentator` exposes default HTTP metrics on `/metrics` (latency histograms, request counters by handler/status).
- Prometheus scrapes `/metrics` (config in monitoring/prometheus.yml).
- Grafana auto-provisions a dashboard with request-rate, latency p95, and total-requests panels (monitoring/grafana/dashboards/heart-api.json).

Screenshots: screenshots/grafana.png, screenshots/prometheus.png.

---

10. Architecture Diagram

```
                  ┌──────────────────────┐
                  │ UCI Heart Disease    │
                  │ (Cleveland CSV)      │
                  └──────────────────────┘
                            │ src/download_data.py
                            ▼
                ┌──────────────────────┐
                │ data/heart_disease_*.csv│
                └──────────────────────┘
                            │ src/train.py (GridSearchCV + MLflow)
                            ▼
┌──────────┐    ┌──────────────────────┐
│ MLflow   │◄──►│ models/              │
│ tracking │    │ heart_disease_model. │
│ (./mlruns)│   │   joblib (Pipeline)  │
└──────────┘    └──────────────────────┘
                            │ FastAPI (src/api.py)
                            ▼
                  ┌──────────────────────┐
                  │ Docker image         │
                  │ heart-disease-api    │
                  └──────────────────────┘
                            │ kubectl / helm
                            ▼
┌──────────────────────────────────────────┐
│ Kubernetes (Deployment + Service + Ingress) │
│     │              │              │       │
│     ▼              ▼              ▼       │
│  /predict       /metrics       /health    │
└──────────────────────────────────────────┘
          │ scrape                 │ logs
          ▼                        ▼
     Prometheus ──────► Grafana   stdout / kubectl logs
```

---

11. Repository Link & Deliverables

- **Code:** https://github.com/uday121212/mlops-heart-disease
- **CI/CD:** `.github/workflows/ci.yml`
- **Helm chart:** `helm/heart-api/`

- **K8s manifests:** `k8s/`
- **Tests:** `tests/`
- **Notebooks:** `notebooks/`
- **Screenshots:** `screenshots/`

---

12. Production-Readiness Checklist

- [x] All scripts run from a clean venv via `requirements-dev.txt`.
- [x] Model serves correctly inside Docker (smoke-tested in CI).
- [x] CI fails fast on lint/test/training errors, with logs.
- [x] Preprocessing is bundled with the model (no train/serve skew).
- [x] Health, metrics, and logging endpoints are exposed.
- [x] K8s probes, resource limits, and non-root container.